

FIA/P GRADUAÇÃO

DOMAIN DRIVEN DESIGN

Prof. Me. Thiago T. I. Yamamoto

#11 – COLLECTIONS FRAMEWORK



TRAJETÓRIA



Collections Framework

#11 - AGENDA



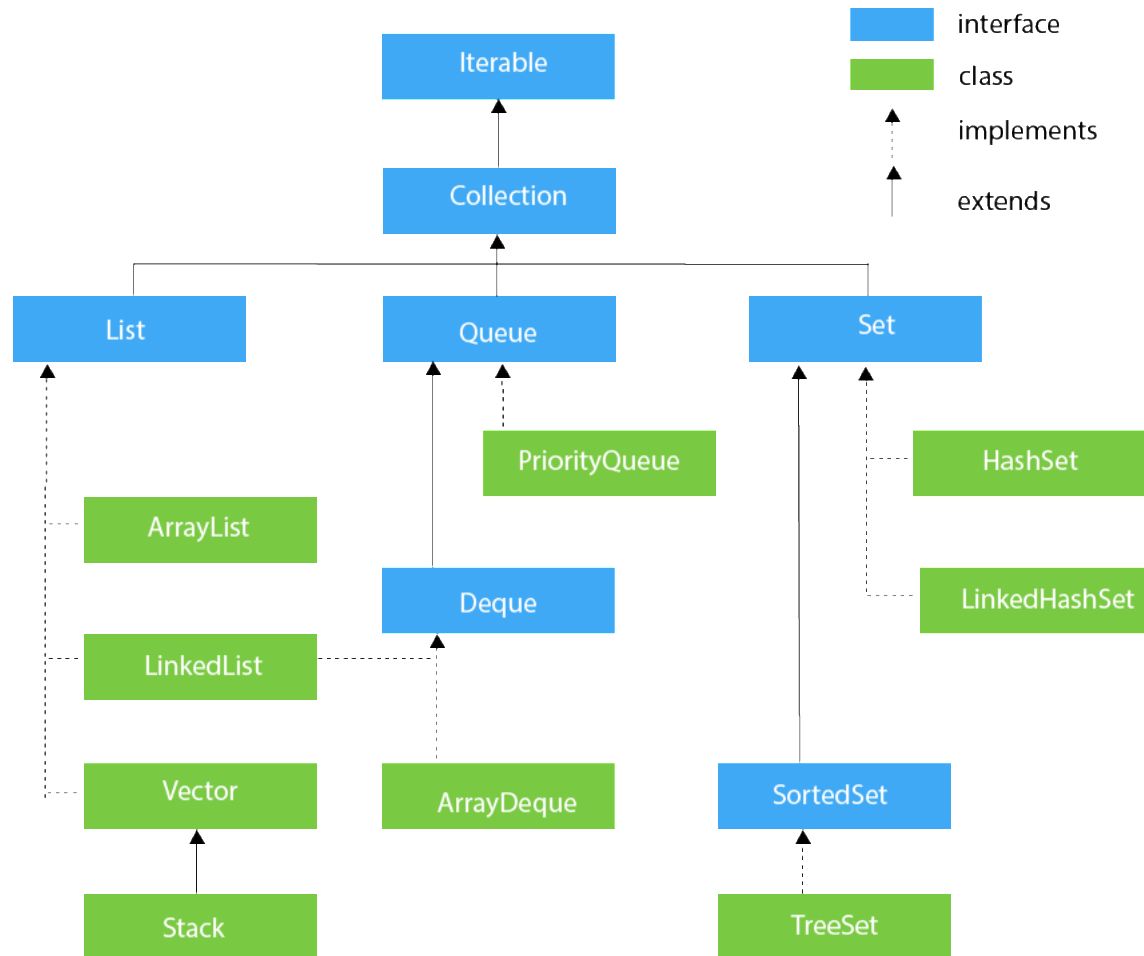
- Collections Framework
- Hierarquia de Classes
- Tipos de Coleção
- Interface Collection
- List
- Set
- Map
- Generics
- Coleções Type Safe
- Autoboxing e Unboxing
- Classes Wrapper

- **Coleções** são **estruturas de dados**, utilizadas para **agrupar objetos**, que permitem de **maneira eficiente e prática** o armazenamento e organização de **objetos**;
- De forma **contrária ao vetor(array)**, elas permitem que um número **arbitrário (não fixo)** de objetos seja armazenado numa estrutura;
- **Coleções** podem ser utilizadas para representar **vetores, listas, pilhas, filas, mapas, conjuntos** e outras estruturas de dados.
- A **escolha de um tipo de estrutura** depende dos requisitos do **problema** que se deseja resolver.

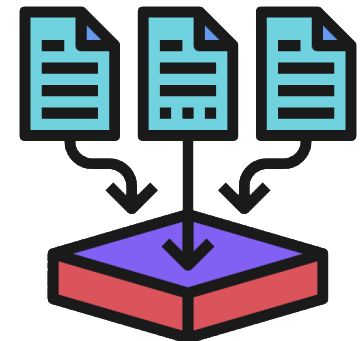
- Várias aplicações necessitam utilizar as **coleções de objetos**, por **exemplo**: agendas pessoais, catálogos de bibliotecas, cadastro de funcionários;
- São amplamente utilizadas no **acesso a dados em bases de dados**, principalmente no **resultado de buscas**;

- As **coleções** são definidas por meio de **interfaces**;
- Como as **interfaces** apenas fornecem **um contrato entre o implementador e os clientes em relação às funcionalidades** de uma abstração, toda **implementação** de uma coleção é realizada numa classe concreta;
- Algumas das **principais interfaces** usadas em coleções são:
 - **Collection**
 - **Set**
 - **List**
 - **Map**

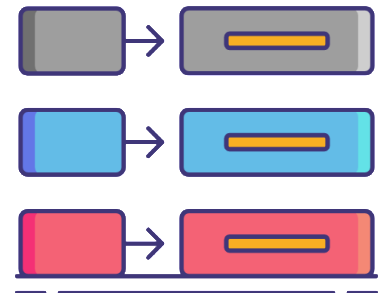
HIERARQUIA DE CLASSES



- No **Java**, as **coleções** podem ser classificadas em **duas categorias**: as que implementam a interface **Collection** e as que implementam a interface **Map**;
- **As subinterfaces principais de Collection são:**
 - **List** - representa uma lista de objetos.
 - Implementação mais utilizada: **ArrayList**.
 - **Set** - representa um conjunto de objetos únicos (os objetos não podem se repetir).
 - Implementação mais utilizada: **HashSet**.



- A interface **Map** representa uma tabela Hash, que **guarda valores compostos por [chave, valor]**, sua principal subinterface é:
 - **SortedMap** - representa um mapa ordenado;
 - Implementação mais utilizada: **HashMap**;



- Denominador comum de **todas as coleções**:
 - **Não há implementações** “diretas” dessa interface;
- Implementações dos tipos **básicos de coleções** tem construtores que recebem um parâmetro do tipo **Collection**;
- **Permite a criação de uma coleção contendo inicialmente todos os elementos da coleção especificada como parâmetro**, independentemente de seu tipo ou implementação;

Método	Descrição
add	Adiciona um objeto a coleção
clear	Remove todos objetos da coleção
contains	Verifica se a coleção contém o objeto determinado
isEmpty	Verifica se a coleção está vazia
remove	Remove um objeto da coleção
size	Retorna a quantidade de objetos na coleção
toArray	Retorna um array contendo os elementos da coleção



LIST

- **Representa uma sequência de elementos** - ordered collection - pode conter elementos **duplicados**;
- Permite controlar a **posição de inserção** de um elemento e acessar elementos por sua posição;
- Permite procurar por um **objeto específico na lista e retornar sua posição numérica**;
- A classe **ArrayList** implementa esta interface;

Método	Descrição
add*	Adiciona um objeto numa determinada posição
get	Retorna o objeto localizado numa determinada posição
remove*	Remove um objeto localizado numa determinada posição
set	Coloca um objeto numa determinada posição (substitui objetos)
indexOf	Retorna a posição de um objeto na lista
lastIndexOf	Retorna a última posição de um objeto na lista
subList	Retorna parte de uma lista

**método sobrecarregado*

- Implementação da interface **List**;
- Armazena seus elementos em um **array** que cresce **dinamicamente**;
- Ótimo **desempenho para acesso em listas** sem muitas modificações no início e meio;

```
ArrayList lista = new ArrayList();  
lista.add("A");  
lista.add("C");  
lista.add("B");  
lista.remove(1);  
System.out.println(lista.get(1)); //B
```



SET

- Uma coleção que **não pode conter elementos duplicados**;
- Corresponde à abstração de um **conjunto**;
- Contém somente **os métodos herdados** da interface **Collection**;
- A classe **HashSet** implementa esta interface;

Método	Descrição
add	Adiciona um objeto no Set
clear	Remove todos objetos do Set
contains	Verifica se o Set possui um objeto determinado
isEmpty	Verifica se o set está vazio
remove	Remove um objeto do Set
size	Retorna a quantidade de objetos no Set
toArray	Retorna um array contendo os objetos do Set

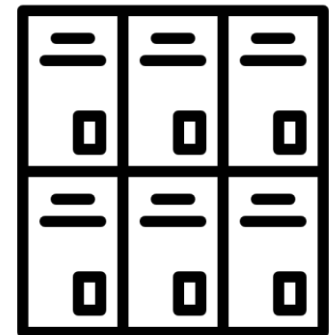
- Implementação da interface **Set**.
- Armazena seus elementos em uma **tabela *hash***;
- Elimina **automaticamente objetos duplicados** inseridos na coleção;

```
HashSet set = new HashSet();  
set.add("um");  
set.add("dois");  
set.add("tres");  
set.add("dois"); //será eliminado da coleção  
set.add("quatro");  
set.remove("tres");  
System.out.println("Tamanho do set: " + set.size());
```



MAP

- Representa um objeto que **mapeia chaves em valores**;
- Um objeto **Map não** pode conter **chaves duplicadas**;
 - cada chave é **mapeada para um único valor**;
- A classe **HashMap** implementa esta interface;



Método	Descrição
clear	Remove todos os mapeamentos
containsKey	Verifica se uma chave já está presente no mapeamento
containsValue	Verifica se um valor já está presente no mapeamento
get	Retorna o valor associado a uma chave determinada
isEmpty	Verifica se o mapeamento está vazio
keySet	Retorna um Set contendo as chaves
put	Adiciona um mapeamento
remove	Remove um mapeamento
size	Retorna o número de mapeamentos
values	Retorna uma Coleção contendo os valores dos mapeamentos

- Implementação da interface **Map**;
 - as entradas são armazenadas em uma tabela *hash*;
 - permite o uso de valores **null**;
- **Não** garante ordenação;

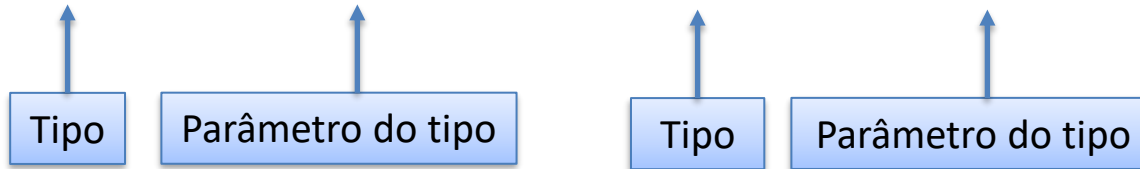
```
HashMap map = new HashMap();  
map.put("1", "um");  
map.put("2", "dois");  
map.put("3", "tres");  
map.put("4", "quatro");  
map.remove("4");  
System.out.println(map.get("1"));
```



GENERIC

- Um **tipo genérico** é um tipo que possui um ou mais **parâmetros de tipo**;
- Recurso introduzido na versão **1.5 do Java**;
- Sintaxe que permite **restringir os tipos de dados** aceitos por referências genéricas;
- Permite a verificação de tipos em tempo de **compilação**;
- Deixa a sintaxe mais limpa, pois elimina a **necessidade de utilizar cast**;
- **Simplicidade**: não é necessário coerção(type casting) para **manipular os objetos extraídos das coleções**;
- **Robustez**: o compilador não permite colocar na coleção elementos **incompatíveis com os tipos declarados**, a **verificação de tipo** é realizada em tempo de **compilação**;

```
ArrayList<Produto> lista = new ArrayList<Produto>();
```



```
ArrayList<Produto> lista = new ArrayList<>();
```

```
ArrayList lista = new ArrayList();
```

```
Produto produto1 = new Produto();
```

```
lista.add(produto1); //[0]
```

```
Produto produto2 = (Produto) lista.get(0);
```

```
Fornecedor fornecedor = new Fornecedor();
```

```
lista.add(fornecedor); //[1]
```

```
Produto produto3 = (Produto) lista.get(1); //exception
```

Exceção: java.lang.ClassCastException

```
ArrayList<Produto> lista = new ArrayList<Produto>();
```

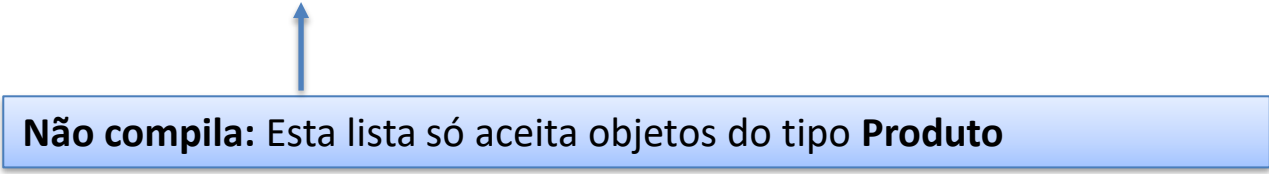
```
Produto produto1 = new Produto();
```

```
lista.add(produto1); //[0]
```

```
Produto produto2 = lista.get(0); //sem cast
```

```
Fornecedor fornecedor = new Fornecedor();
```

```
lista.add(fornecedor); // não compila!
```



Não compila: Esta lista só aceita objetos do tipo **Produto**

- Toda a **API de coleções** foi adaptada para permitir o uso de tipos **genéricos**;
- **Maior segurança** nas checagens de tipos;
- Não há mais necessidade de **cast**;
- **Sem os genéricos**, o programador precisava lembrar o tipo de objeto guardado na coleção;
- O código fica um **pouco mais complexo** no momento de declarar o tipo parametrizado, mas apenas em um ponto;



- Tipos genéricos **não podem** ser parametrizados com tipos **primitivos**:

`ArrayList<int>` ou `HashSet<char>` são inválidos!

- Declaração de **coleções com genéricos**:

```
ArrayList<String> listaString = new ArrayList<String>();  
HashSet<Produto> setProduto = new HashSet<Produto>();  
HashMap<String, Produto> mapProduto = new HashMap<String, Produto>();
```

```
ArrayList<String> lista = new ArrayList<String>(); //Somente String

String nome = new String("Gabriela");

lista.add(nome);

lista.add(new String("Bruno"));

lista.add("Thiago");

//lista.add(1); //Não compila, a lista só aceita String

String busca = lista.get(1); //Não precisa de cast

for (String s : lista) {

    System.out.println(s);

}
```



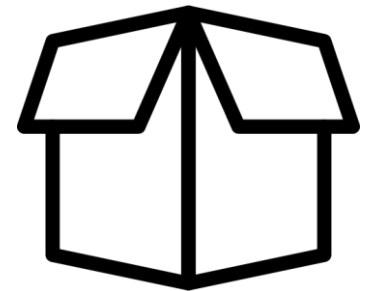
AUTOBOXING E UNBOXING

- **Autoboxing:** conversão automática de tipos primitivos para objetos das classes wrapper;

```
Integer x = 10;
```

- **Unboxing:** conversão automática de objetos das classes wrapper em tipos primitivos;

```
double pi = new Double(3.14);
```



Tipo	Tipo Primitivo	Classe Wrapper	Super Classe
Lógico	boolean	Boolean	Object
Caractere	char	Character	
Inteiro	byte	Byte	Number
	short	Short	
	int	Integer	
	long	Long	
Real	float	Float	
	double	Double	

- **Autoboxing**, exemplos de aplicação em coleções:

```
HashMap<Integer, Double> mapa = new HashMap<Integer, Double>();
```

```
mapa.put(3,3.45); //autoboxing: int > Integer, double > Double
```

```
ArrayList<Integer> lista = new ArrayList<Integer>();
```

```
lista.add(1); //autoboxing: int > Integer
```

```
lista.add(2); //autoboxing
```

```
lista.add(new Integer(3)); //sem autoboxing, Integer é objeto
```

```
lista.add(8); //autoboxing
```

- **Unboxing**, exemplos de aplicação em coleções:

```
double valor = mapa.get(3); //unboxing: Double > double
```

```
int primeiroValor = lista.get(0); //unboxing: Integer > int
```



PRÁTICA



1. Crie um **programa em Java** que permita ao usuário adicionar quantos alunos desejar à lista de alunos.
 - a) Depois que o usuário acabar de inserir os alunos, **mostrar na tela:**
 - i) Total de alunos cadastrados
 - ii) Média geral de notas
 - iii) Média geral de idade
 - iv) Maior Idade
 - v) Menor Idade

- b) Após exibir as informações descritas anteriormente, o sistema deverá apresentar um menu ao usuário que possibilite a ele executar as seguintes funcionalidades:
- i) Imprimir os nomes e médias de todos os alunos
 - ii) Imprimir apenas os nomes dos alunos que possuem média acima de 6.0
 - iii) Imprimir apenas os nomes dos alunos que possuem idade acima de 30 anos
 - iv) Excluir os alunos com médias menores que 3.0
 - v) Sair do sistema

Nota: 1) Para a implementação deste programa utilize sempre que possível objetos das classes ArrayList, HashSet e HashMap. 2) O aluno possui os atributos: RM, Nome, Idade, Nota1 e Nota2.

2. Desenvolva um **programa em Java** que permita ao usuário cadastrar e consultar os números de DDD pertencentes a um estado(UF).

- a) Ao ser executado o sistema deverá exibir um menu ao usuário onde é apresentado 3 opções:
 - i) Incluir o estado e seus DDDs
 - ii) Consultar os DDDs de um estado
 - iii) Sair
- b) Caso o usuário selecione a opção 1, o sistema deverá solicitar a sigla do estado e depois permitir que o usuário atribua quantos números de DDD desejar.
- c) Caso o usuário selecione a opção 2, o sistema deverá solicitar a sigla do estado e depois exibir os números de DDD que foram atribuídos a ele.

Copyright © 2020 - 2025
Prof. Thiago T. I. Yamamoto

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).

“A felicidade não está em fazer o que a gente quer, e sim querer o que a gente faz”. (Jean Paul Sartre)